
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON CHOIR - Requirements & Grading Criteria

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

OPERATION IRON CHOIR

REQUIREMENTS & GRADING CRITERIA

TARGET: NorthPharma factory floor andon station (line edge controller)

ARTIFACT: ACT-VII.bin / ACT-VII.uf2 (compromised)

CREW: FROSTLINE

OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma runs the line that makes the medicine, and its factory floor andon station built on a Pico 2 is the node on the edge of that line. A contractor called **FROSTLINE** planted an implant in the node image: a raw-frame command-and-control check-in that registers with a local listener using the `C2V1` magic and bot id `0xB7`, a benign remote task handler, a reserved-sector bot marker that re-arms the bot on every boot, and an inverted fault-clear authorization verdict. Operative **NIGHTINGALE** recovered the compromised image as `ACT-VII.bin`.

Students are the reverse-engineering reserve. They reverse engineer `ACT-VII.bin` with Ghidra, find and patch all four defects, defeat the CoreDebug `DHCSR` anti-debug under GDB to observe the marker write, export a corrected image, flash it to a real Pico 2, and prove the corrected behavior on the breadboard. The machine check is `scripts/verify_ctf.py`.

The challenge is a standalone capstone exercise and contains no answer, constant, address, bug, or patch belonging to any other course assignment.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the monitor loop.
- Locate four corrupted bytes: a command-and-control check-in gate, a remote task gate, a reserved-sector marker gate, and an authorization verdict branch.
- Analyze `cbz` and `cbnz` condition semantics and branch inversion.
- Explain why a raw pre-authentication payload path is invisible to a sealed protocol and why a check-in never needs the cipher.
- Explain why an obedience channel that runs only benign tasks is still a control-system failure.
- Explain why reserved-flash state survives a firmware reflash.
- Read CoreDebug `DHCSR`, explain the anti-debug trap, and defeat it under GDB.
- Explain why authentication is not authorization and why a verdict must be verified before the command is applied.

Students must use only the course concepts: ARM registers, stack behavior, USB-CDC and UART consoles, GDB, Ghidra static analysis and binary patching, vector tables, reset startup, XIP, Thumb addressing, condition-code analysis, stateful security, and the Argon2id plus XChaCha20-Poly1305 authenticated envelope.

Deliverables Checklist

#	Deliverable	Format	Criterion
1	Ghidra project screenshot	PNG/JPG	Task 1
2	Vector table and boot table	Inside <code>ACT-VII-Answers.md</code>	Task 1
3	<code>main</code> and monitor-loop table	Inside <code>ACT-VII-Answers.md</code>	Task 1
4	Module map	Inside <code>ACT-VII-Answers.md</code>	Task 1
5	C2 check-in evidence and patch	Inside <code>ACT-VII-Answers.md</code>	Task 2
6	Task handler evidence and patch	Inside <code>ACT-VII-Answers.md</code>	Task 3
7	Anti-debug GDB proof, reserved-sector evidence, and patch	Inside <code>ACT-VII-Answers.md</code>	Task 4
8	Fault-clear authorization evidence and patch	Inside <code>ACT-VII-Answers.md</code>	Task 5
9	<code>ACT-VII_fixed.bin</code>	BIN file	Task 6
10	<code>ACT-VII_fixed.uf2</code>	UF2 file	Task 6
11	Hardware proof and reflection	Inside <code>ACT-VII-Answers.md</code>	Task 6

Required Tools and Equipment

Tool	Purpose
Raspberry Pi Pico 2	Isolated target node
Debug Probe (OpenOCD)	SWD connection for GDB inspection and the anti-debug work
<code>arm-none-eabi-gdb</code>	Runtime breakpoints, <code>DHCSR</code> clearing, and reserved-sector observation
Ghidra	Static analysis and binary patching
Python 3 with <code>uf2conv.py</code>	UF2 conversion and artifact checks
DHT11, 1602 I2C LCD, RYL998, IR receiver, SG90 servo, 3 LEDs, fault-clear button	Breadboard hardware proof
<code>ACT-VII.bin</code> and <code>ACT-VII.uf2</code>	Supplied compromised artifacts

Console settings: **USB-CDC virtual COM port, 115200 baud, 8 data bits, no parity, 1 stop bit.** Radio UART settings: **UART1, 115200, network ID 18.**

Artifact Identity

The instructor-issued artifact hashes are:

ACT-VII.bin	c9996a62a7157e7aed43b3caf4e33348a87b8adfa4bbf67e1f1ff94df89cef42
ACT-VII.uf2	79b92868b5218572dfe41555e5b5157a01f1f277724ac5d7524de657f2def8e7
ACT-VII_fixed.bin	b5fe79ce517f820449ad1dc2ecf628e6702891174abc9c33e923556d7f8c7534
ACT-VII_fixed.uf2	6aa0f6742ae2baf8ab44f68a6d5bb7d26fa19901f3add2bc2a0311ddabd442f4

The verifier checks the `ACT-VII.bin` and `ACT-VII_fixed.bin` hashes specifically, asserts the four fixed bytes, and requires that only those four offsets differ between the two `.bin` images. Both `.bin` images are 51,196 bytes and both `.uf2` images are 102,912 bytes.

Grading Rubric - Detailed Breakdown

Task 1: Setup and Initial Analysis (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronChoir_Investigation</code> , raw binary import	One item off	Not set up
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor	Wrong language	Missing
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>	Wrong base	Missing
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>	One missing	Not found
[DOCUMENT] main and the andon monitor state machine (monitor_step) addresses identified	1	main <code>0x10000234</code> , monitor_step <code>0x100065CC</code>	One correct	Neither
[DOCUMENT] Module map identifies the diverter, control, andon_auth, implant, and monitor anchors	1	At least one correct anchor per module	Partial	Missing

Task 2: Bug #1 The C2 Check-In (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the C2 check-in branch at 0x1000A47F	5	Address and function (<code>implant_tick</code> , inlined <code>implant_check_in</code>) identified	Approximate	Not found
[DOCUMENT] Documented the C2V1 check-in frame and the bot id 0xB7	5	4-byte <code>C2V1</code> magic, bot id <code>0xB7</code> , six-byte frame, four-tick interval	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the node does not register	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained why the raw check-in needs no cipher and hides beneath the sealed path	3	Raw path under the sealed envelope and a listener that holds no key	Vague	Missing

Task 3: Bug #2 The Task Handler (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the task handler branch at 0x1000A2A9	5	Address and function (<code>implant_handle_command</code> , inlined <code>implant_tasking_frame</code>) identified	Approximate	Not found
[DOCUMENT] Documented the TASK_BLINK, TASK_LOG, TASK_REPORT handler	5	Magic match before the sealed path, three benign tasks, unknown task ignored	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so no remote task executes	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained why stopping the task handler is separate from stopping the check-in	3	The handler obeys, the check-in announces	Vague	Missing

Task 4: Bug #3 The Bot Marker (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the bot marker branch at 0x1000A3EF	5	Address and inlined <code>implant_init</code> path identified	Approximate	Not found
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so no marker is written to 0x103FF000	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained the reserved sector 0x103FF000 and the bot marker byte 0xC7	3	Marker, reserved sector, write-once first run	Vague	Missing

Task 5: Bug #4 The Fault-Clear Authorization (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the fault-clear authorization branch at 0x10007541	5	Address and function (<code>control_handle_frame</code>) identified	Approximate	Not found
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so failed and replayed authorizations are rejected	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Explained why an unauthenticated or replayed fault or clear envelope must be rejected	3	The applied command must see only an authorized verdict	Vague	Missing

Task 6: Export and Verify (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[PATCH] Exported ACT-VII_fixed.bin from Ghidra	2	Valid patched binary	Corrupt	Not submitted
[PATCH] Converted to ACT-VII_fixed.uf2 with the correct base and family	2	<code>--base 0x10000000 --family 0xe48bff59</code>	Wrong flags	Not submitted
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown	Partial proof	No proof
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four	Partial	Missing

Common Pitfalls

Pitfall	Consequence	Avoidance
Reading the check-in gate backwards	The node still registers with the C2 listener	Suppress only on the clear-gate branch (<code>cbz</code> , <code>0xB1</code>)
Reading the task gate backwards	The node still executes a remote task	Neutralize only when the gate is clear (<code>cbz</code> , <code>0xB1</code>)
Confusing <code>cbz</code> and <code>cbnz</code> at <code>0xA3EF</code> or <code>0x7541</code>	The marker is still written, or a failed authorization is still accepted	Neutralize only when the gate or verdict is clear (<code>cbz</code> , <code>0xB1</code>)
Patching the low byte at <code>0xA386</code> , <code>0xA1B0</code> , <code>0xA2F6</code> , or <code>0x7444</code>	The condition code never changes	Patch the high byte at <code>0xA47F</code> , <code>0xA2A9</code> , <code>0xA3EF</code> , <code>0x7541</code>
Searching for a standalone <code>implant_infect</code> , <code>implant_check_in</code> , or <code>implant_tasking_frame</code> symbol	Cannot find the inlined gates	Look inside <code>implant_init</code> at <code>0xA3EF</code> , <code>implant_tick</code> at <code>0xA47F</code> , and <code>implant_handle_command</code> at <code>0xA2A9</code>
Confusing the check-in with the task handler	Both gates sit in the implant, at <code>0xA47F</code> and <code>0xA2A9</code>	Patch the check-in gate in <code>implant_tick</code> first, then the task gate
Patching the shipped image before observing the write	You never prove the bot marker write	Defeat <code>DHCSR</code> under GDB first, then patch the artifact
Fabricating the GDB session	Verification fails	Show the command sequence and the real observed code path

Pitfall	Consequence	Avoidance
Treating the anti-debug as a defect to patch	Wasted effort; it is identical in both images	Defeat it in a scratch copy or with GDB, then patch the real defect
Missing that the authorization branch is a verdict	Unauthenticated commands still reach the applied command and zone	Accept only when the verdict is true (<code>cbz</code> to reject, <code>0xB1</code>)
Forgetting UF2 conversion	Raw binary will not flash	Use <code>uf2conv.py</code> with family <code>0xe48bff59</code>

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 line temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Line diverter servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	FAULT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	ACK PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	LINE OK, 220 to 330 ohm to GND
Fault-clear button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
CoreDebug_DHCSR	0xE00EDF0	Anti-debug register read by the implant
Implant reserved sector	0x103FF000	Bot marker target (last flash sector)
Implant tick counter	0x20013714	Incremented once per <code>implant_tick</code>
Implant task count	0x20013710	Number of executed remote tasks
Implant check-in count	0x2001370C	Number of emitted check-in frames
Implant armed flag	0x20013CF5	Set when the payload handler arms
Implant checked-in flag	0x20013CF6	Set after a check-in frame is emitted
Implant check-in enable	0x20013CF7	Enables the check-in readiness test
Implant check-in gate	0x20013CF8	Gates the <code>C2V1</code> registration
Implant last task	0x20013CF9	Last executed task code
Implant marker gate	0x20013CFA	Gates the reserved-sector marker write
Implant task gate	0x20013CFB	Gates the remote task handler
Implant tasking enable	0x20013CFC	Enables the tasking readiness test
Control ready gate	0x20013CF1	Gates the sealed fault command path
Applied command	0x20013CF0	Command after a true verdict
Applied zone	0x20013CE2	Zone after a true verdict
Authorization ready gate	0x20013CEC	Gates the authorization check
Auth state record	0x200136CC	Anti-replay and state-tag record
Control field key	0x200131F4	Derived field key for the envelope
Envelope workspace	0x200136E8	Sealed frame open workspace

The VA of any file offset is the file offset plus `0x10000000`.

Deadline & Submission

- Create a folder containing the Ghidra screenshot, `ACT-VII_fixed.bin`, and `ACT-VII_fixed.uf2`.
- Write all written answers in `ACT-VII-Answers.md` inside that folder.
- Include the output of `python scripts/verify_ctf.py`.

- ZIP the folder as `lastname-firstname-ACT-VII.zip`.
- Submit the ZIP before the posted deadline; late submissions lose 10 percent per day.

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86
B-	80-83%	80-83
C	70-79%	70-79
F	0-69%	0-69%

Academic Integrity

Use only the supplied Pico 2 and firmware. Do not connect the exercise to an operational factory network, a manufacturing execution system, a building-management system, a public network, a military system, or a third-party device. This is a controlled, isolated educational exercise. All analysis and patches must be your own work; sharing binaries, addresses, keys, passphrases, or answers is a violation of the academic integrity policy.

Reference Material

Topic	Reference
ARM Cortex-M33 registers and stack	Course block 1
USB-CDC and UART console capture	Course block 2
Vector tables, reset startup, and XIP	Course block 3
Ghidra static analysis and binary patching	Course block 4
Raw control payloads, magic preambles, and pre-authentication surface	Course block 5
Command-and-control check-in, bot identifiers, and fixed-cadence registration	Course block 6
Reserved-flash persistence and boot re-install	Course block 7
CoreDebug <code>DHCSR</code> and anti-debug	Course block 8
Argon2id and XChaCha20-Poly1305 authenticated envelope	Course block 9